

Book Worm e-bookstore backend

A Spring Boot 3 and PostgreSQL REST API for an online bookstore, covering all 12 customer journeys in the brief. Designed spec first with Claude Code, built with IBM Bob, and verified end to end against the real database.

12 / 12 customer journeys built	25 API operations in the OpenAPI contract	146 automated tests, 0 failures	40 / 40 live journey calls on PostgreSQL
5 design specs written before the code	13 tables, 2 Flyway migrations	33 entries in the AI usage log	2 agentic tools: Claude Code and IBM Bob

Author	Ed Duenas
Date	8 October 2026
Repository	github.com/laharl143/ebookstore-backend
Pull request	github.com/laharl143/ebookstore-backend/pull/1 (branch <code>feature/api-implementation</code>)
Stack	Java 17, Spring Boot 3.5.16, Maven, PostgreSQL 16, Flyway, Spring Security (JWT), springdoc
Evidence	<code>docs/ai-usage-log.md</code> , <code>docs/specs/</code> , <code>docs/screenshots/</code> , the walkthrough video

Contents: 1. Scope and approach · 2. How AI was used · 3. Architecture and data model · 4. API design · 5. Proof that it runs · 6. Testing and the bugs live runs caught · 7. Lessons and limits

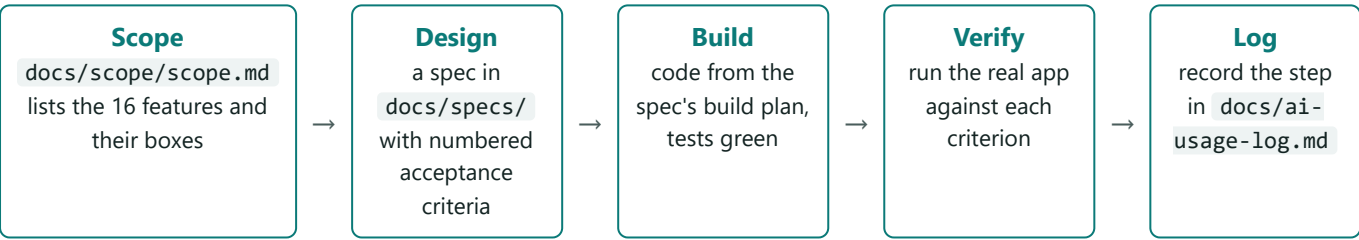
1. Scope and approach

The brief asks for the backend of an e-commerce bookstore: customers browse, choose and buy books. I aimed for the smallest build that meets every review criterion (agentic IDE use, API design, a working app, best practices), so the MVP is the 12 customer journeys on the use case slide, each built thin. Wishlist, reviews, coupons, admin, shipping and Docker were moved to a Deferred list.

Phase	Features	Status
Foundations	Stack and architecture · coding standards · data model · API contract · authentication	done
Journey 1: Catalogue	Browse, search and filter · brands (authors, publishers) · book detail with delivery date · related books	done
Journey 2: Payment and purchase	Basket · checkout with server side totals and gift points · simulated payment and confirmation · cancel within 48 hours	done
Journey 3: E-store Home	Order history and Buy It Again · recommendations from order history	done
Submission	Run kit (README, Insomnia) · branch and pull request · video and this report	video last

Build approach

I worked **journey by journey**: finish one customer journey from the slide end to end, then start the next. Every feature follows the same loop, kept in the repo so both AI tools share one source of truth:



Project rules live in AGENTS.md, which both tools load: a layered monolith (controller, service, repository), constructor injection, record DTOs, named exceptions turned into RFC 9457 errors by one handler, Flyway only for schema changes, the OpenAPI file as the contract, and no secrets in git.

Choices that differ from the brief

- **Philippine pesos and Philippine addresses** (4 digit ZIP, +63 phone, province) instead of the rupees and Indian addresses in the wireframes.
- **Payment methods** are credit card, debit card and e-wallet; UPI was dropped as it does not fit a Philippine store.
- **Local run only**, which the brief allows; cloud deployment was deferred.

2. How AI was used

I used two agentic tools and gave each a clear job. Every prompt, output and decision is in `docs/ai-usage-log.md` (33 entries), each naming the tool that ran it.

Tool	What it did
Claude Code desktop app, Claude Opus	Turned the brief into a scope; designed specs 0001 (stack), 0002 (data model) and 0003 (API contract), asking me each design question with a recommended answer; built the stack scaffold and the data model (2 migrations, 12 entities); set up the IBM Bob workflow; ran the end to end verification, found the bugs, and checked Bob's fixes; wrote the PR description, video script and this report.
IBM Bob VS Code based IDE	Built features 4 to 15 from the specs: the OpenAPI file and error handling, authentication (spec 0004), the catalogue, basket, checkout (spec 0005), payment, cancel, history, Buy It Again and recommendations, plus the README and Insomnia collection. Fixed the bugs found in the end to end runs, with regression tests.

Making two tools work as one team

- **Shared rules and state in the repo.** Both tools read `AGENTS.md`, the scope and the specs, so either can pick up where the other stopped.
- **A Bob workflow I wrote with Claude Code:** `.bob/rules/01-workflow.md` plus four slash commands, `/architect`, `/develop`, `/verify` and `/sync`, mirroring the Claude Code skills. `/develop` refuses to build without a spec and stops to ask when a value has no named source, instead of inventing one.
- **Tuned for flow:** an auto approve list for safe commands (with a deny list for force push and hard reset), a rule to run one plain command per call, and a sound hook so I hear when Bob finishes.

Keeping the AI honest

Each spec was **cross checked by a second model** before it was accepted. The reviews found 28 issues in the stack spec, 21 in the data model and 25 in the API contract, for example SQL types that would fail on the test database, two checkouts at once creating two orders, and a declined payment record that would be rolled back. Every fix was applied before any code was written.

What I decided, and what the AI decided. I answered the design questions that matter for the business (pesos and Philippine addresses, VAT on the subtotal then points, one row per book format, workflow depth, where to skip verification). For lower level choices I often told the tool to "pick the recommended option"; each of those is recorded in the spec's decision log so I can review or override it. The AI never chose a business rule silently.

3. Architecture and data model

Stack (spec 0001)

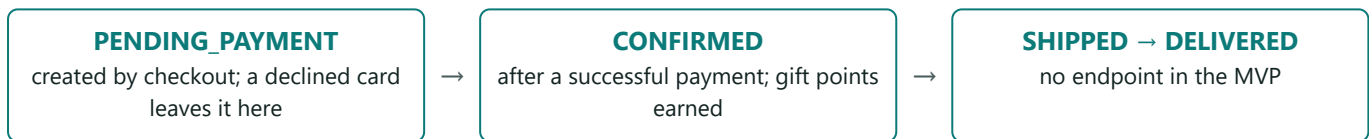
Layer	Choice
Pattern	Layered monolith: controller → service → repository
Runtime	Java 17, Spring Boot 3.5.16, Maven wrapper
Database	PostgreSQL 16; H2 in PostgreSQL mode for tests
Schema	Flyway SQL migrations; Hibernate only validates
Security	Spring Security resource server, HS256 JWT (60 min), BCrypt
Errors	One handler, RFC 9457 Problem body with a fixed <code>code</code>
Docs	Hand written <code>openapi.yaml</code> , Swagger UI via springdoc
Secrets	Env vars or a gitignored local file only

Data model (spec 0002), 13 tables

Area	Tables
Customers	<code>users</code> (with gift points balance), <code>addresses</code>
Catalogue	<code>categories</code> , <code>genres</code> , <code>authors</code> , <code>publishers</code> , <code>books</code> (one row per format), <code>book_genres</code>
Buying	<code>cart_items</code> , <code>orders</code> (address and price snapshots), <code>order_items</code> , <code>payments</code> (card last 4 only)
Loyalty	<code>gift_point_transactions</code> (a ledger that always adds up to the balance)

Seed data: 19 categories, 10 genres, 9 authors, 4 publishers, 27 books priced in pesos. 61 named constraints let the database itself reject bad data (negative stock, a total that does not add up, a card payment without its last 4 digits).

Order lifecycle



CANCELLED can be reached from **PENDING_PAYMENT** at any time, or from **CONFIRMED** within 48 hours of placing the order. Cancelling restores stock and redeemed points, reverses earned points and refunds the payment.

Rules the design enforces

- **All money is computed on the server:** subtotal, 12% VAT, delivery charge, points discount and total. The client never sends a price.
- **No double spending:** stock, points and order status change only through guarded single statement updates, and checkout locks the user's row, so two requests at once cannot oversell or pay twice.
- **Card data minimised:** the full card number, CVV and expiry are validated (including a Luhn check) and then discarded.

4. API design

The API is **API first**: spec 0003 fixed every endpoint before the controllers were written, and `src/main/resources/openapi.yaml` (OpenAPI 3.0.3) is the contract. A **drift test** fails the build if any controller serves a path that is not in the file, and another test checks the file is valid.

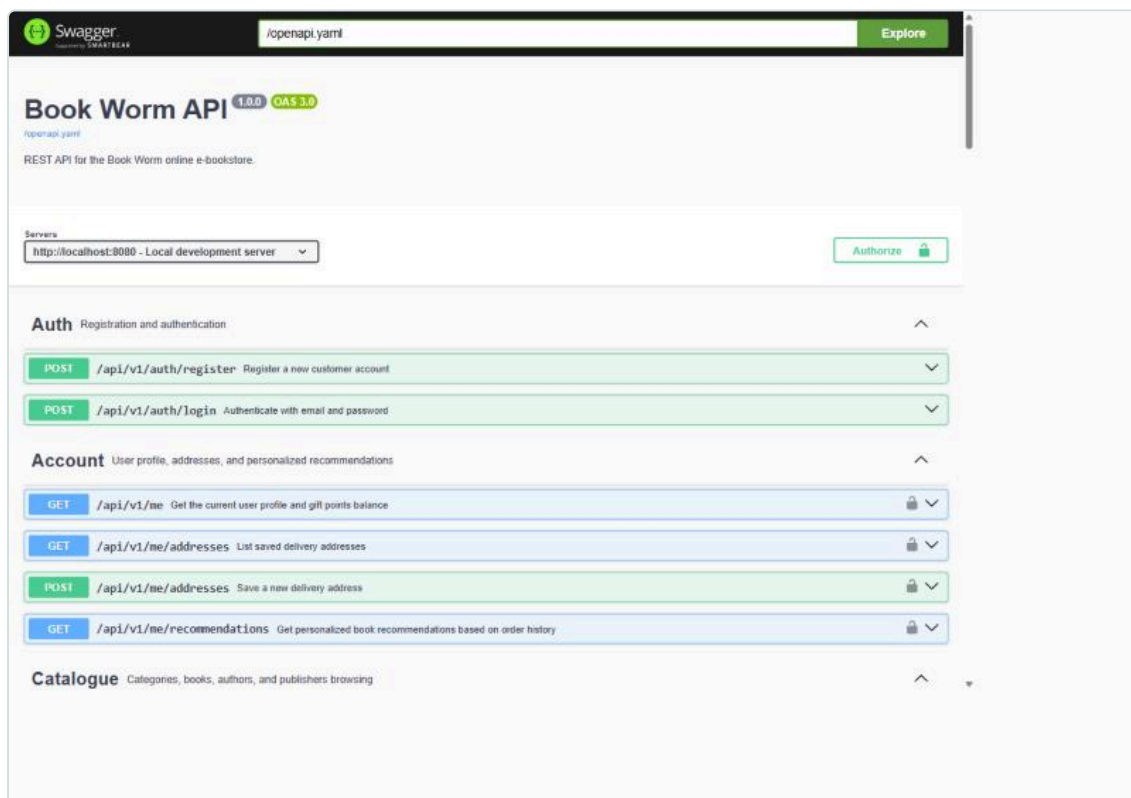
Group	Operations (all under <code>/api/v1</code>)	Auth
Auth	<code>POST /auth/register</code> · <code>POST /auth/login</code>	public
Account	<code>GET /me</code> · <code>GET, POST /me/addresses</code> · <code>GET /me/recommendations</code>	token
Catalogue	<code>GET /categories</code> · <code>GET /books</code> (search, filters, sort, paging) · <code>GET /books/{id}</code> · <code>GET /books/{id}/related</code> · <code>GET /authors</code> · <code>/authors/{id}</code> · <code>GET /publishers</code> · <code>/publishers/{id}</code>	public
Cart	<code>GET /cart</code> · <code>POST /cart/items</code> · <code>PUT, DELETE /cart/items/{bookId}</code> · <code>DELETE /cart</code>	token
Orders	<code>POST /orders</code> · <code>GET /orders</code> · <code>GET /orders/{id}</code> · <code>POST /orders/{id}/cancel</code> · <code>POST /orders/{id}/buy-again</code>	token
Payments	<code>POST /orders/{id}/payments</code>	token

Conventions

- Money as decimals with 2 places plus `"currency": "PHP"`; UTC timestamps; local delivery dates.
- Lists are paged: `page`, `size` (default 12, max 50).
- The customer always comes from the token, never from the URL; another user's order returns 404, so ids cannot be probed.

One error format

- Every error is `application/problem+json` with a stable code, for example `INSUFFICIENT_STOCK`, `PAYMENT_DECLINED` (402), `CANCEL_WINDOW_EXPIRED` (409).
- Validation errors list each bad field.



Swagger UI serving the hand written contract at `/swagger-ui.html`.

5. Proof that it runs

On 8 October 2026 the app was started against the local PostgreSQL database (Flyway applied both migrations) and a script walked every journey as a fresh customer: 40 calls covering sign up and log in, browsing, the basket, checkout, a declined and a successful payment, history, Buy It Again, recommendations and cancel. **All 40 returned the expected result**, including the planned error cases (duplicate email 409, wrong password 401, no token 401, out of stock 409, declined card 402, second cancel 409). The 33 request Insomnia collection in docs/ was also run in full: 33 of 33 as expected.

Book Worm API: every journey, end to end

Live calls against the running app on PostgreSQL 16 at 2026-10-08 10:21 UTC, as a fresh test customer. Red codes are the expected error cases (duplicate email, wrong password, no token, out of stock, and so on). 40 of 40 steps as expected

STATUS	REQUEST	STEP	RESULT
1. HOME: SIGN UP AND LOG IN			
201	POST /auth/register	Register a new customer	token issued (expires in 3600s) for s2b-2026100810215@bookworm.test
409	POST /auth/register	Register the same email again	EMAIL_TAKEN: An account with email 's2b-2026100810215@bookworm.test' already exists
401	POST /auth/login	Log in with a wrong password	INVALID_CREDENTIALS: Invalid email or password
200	POST /auth/login	Log in	token issued (expires in 3600s) for s2b-2026100810215@bookworm.test
401	GET /me	Protected call without a token	UNAUTHORIZED: Full authentication is required to access this resource
200	GET /me	My profile and points	Juan Dela Cruz, 0 gift points
2. CATALOGUE			
200	GET /categories	All categories (sidebar)	10 items: Romance, Mystery, Science Fiction, Fantasy, ...
200	GET /books?category=romance&size=3	Books in Romance	4 found: Emma, Pride and Prejudice, Pride and Prejudice
200	GET /books?q=rizzal&sort=price_asc	Search "rizzal", cheapest first	5 found: Noli Me Tangere, Noli Me Tangere: Salin sa Filipino, Noli Me Tangere, ...
404	GET /books?category=none	Unknown category	CATEGORY_NOT_FOUND: Category not found for slug: none
200	GET /authors?size=5	Browse brands/ authors	9 found: Arthur Conan Doyle, Bram Stoker, R. G. Wills, ...
200	GET /books/1	Book detail with delivery date	Noli Me Tangere (PAPERBACK), PMSB, by José Rizal, delivery 2026-10-11
200	GET /books/1/related?size=4	Related reads	4 items: Noli Me Tangere: Salin sa Filipino, El Filibusterismo, Pride and Prejudice, The Hound of the Baskervilles
404	GET /books/999999	Unknown book	BOOK_NOT_FOUND: Book not found with ID: 999999
200	GET /me/recommendations?size=4	Recommendations for a new customer (newest books)	4 items: Loving Bahay: Filipino Home Cooking, The Invisible Man, Ang Munting Kalabaw, Paper: A Life of José Rizal
3. BASKET			
200	GET /books?size=50	Find more books	27 found: A Study in Scarlet, Ang Munting Kalabaw, Brucala, ...
200	POST /cart/items	Add 2 paperbacks	2 items, subtotal ₱599, VAT ₱71.88, total ₱670.88

Journey run, part 1: sign up and log in, catalogue, basket.

409	POST /cart/items	An out of stock book	INSUFFICIENT_STOCK: Insufficient stock for book ID: 8
200	PUT /cart/items/1	Change quantity with the stepper	2 items, subtotal ₱599, VAT ₱71.88, total ₱670.88
200	GET /cart	View the basket with totals	2 items, subtotal ₱599, VAT ₱71.88, total ₱670.88
4. PAYMENT AND PURCHASE			
201	POST /orders	Place the order (new address, saved)	BU-20261008-000005 PENDING_PAYMENT, total ₱670.88, delivery 2026-10-11
409	POST /orders	Redeem more points than the balance	CART_EMPTY: Cannot place an order with an empty cart
402	POST /orders/5/payments	Pay with a card that is declined	PAYMENT_DECLINED: Card declined
201	POST /orders/5/payments	Pay with a valid card (purchase confirmation)	payment SUCCESS, card ****4342, order BU-20261008-000005 CONFIRMED, +6 points
409	POST /orders/5/payments	Pay the same order again	GROUP_NOT_PAYABLE: Order is not in a payable state
200	GET /orders/5	Order detail after a decline then a success	BU-20261008-000005 CONFIRMED, total ₱670.88, delivery 2026-10-11
200	GET /books/1	Copies sold after paying (120 before, 121 now, should be +1)	Noli Me Tangere (PAPERBACK), PMSB, by José Rizal, delivery 2026-10-11
200	GET /me	Gift points after paying	Juan Dela Cruz, 6 gift points
200	GET /me/addresses	Saved addresses	1 item: Makati
5. HOME: HISTORY AND RECOMMENDATIONS			
200	GET /orders	Order history	[{"items":[{"id":5,"orderNumber":"BU-20261008-000005","status":"COMPLETED","items":[{"bookId":1,"title":"Noli Me Tangere","format":"PAPERBACK"}]}
200	POST /orders/5/buy-again	Buy It Again	added 2, skipped 0
200	GET /me/recommendations?size=4	Recommendations from order history	4 items: Noli Me Tangere: Salin sa Filipino, The Hound of the Baskervilles, El Filibusterismo, The Adventures of Sherlock Holmes
6. CANCEL WITHIN 48 HOURS			
201	POST /orders	Second order with saved address, redeeming 5 points	BU-20261008-000006 PENDING_PAYMENT, total ₱665.88, delivery 2026-10-11
201	POST /orders/6/payments	Pay it with an e-wallet	payment SUCCESS, card ****, order BU-20261008-000006 CONFIRMED, +6 points
200	POST /orders/6/cancel	Cancel it (refund, stock and points restored)	BU-20261008-000006 CANCELLED, total ₱665.88, delivery 2026-10-11
409	POST /orders/6/cancel	Cancel it again	ORDER_NOT_CANCELLABLE: Order cannot be cancelled in status: CANCELLED
200	GET /me	Points after the cancel	Juan Dela Cruz, 6 gift points
200	POST /orders/5/cancel	Cancel the first order (it had a declined attempt)	BU-20261008-000005 CANCELLED, total ₱670.88, delivery 2026-10-11

Journey run, part 2: checkout, declined then successful payment, history, Buy It Again, cancel.

5. Proof that it runs (continued)

The same run, checked directly in PostgreSQL: both orders were cancelled, the declined attempt is kept as FAILED with only the last 4 digits, only the real payment was refunded, and the gift points ledger adds up exactly to the balance.

The same run, in PostgreSQL

psql against the `bookworm` database after the journey run: this run's customer only; both orders cancelled; the declined attempt kept as `FAILED` and only the real payment refunded; a gift points ledger that adds up to the balance; and the saved Philippine address.

```

e2w-2026100810215@bookworm.test
bookworm=> SELECT order_number, status, subtotal, vat_amount, gift_points_redeemed, total_amount, gift_points_earned, cancelled AS NOT NULL AS cancelled FROM orders o WHERE <this customer>
order_number | status | subtotal | vat_amount | gift_points_redeemed | total_amount | gift_points_earned | cancelled
-----
BW-20261008-000005 | CANCELLED | 599.00 | 71.88 | 0 | 670.88 | 6 | t
BW-20261008-000006 | CANCELLED | 599.00 | 71.88 | 5 | 665.88 | 6 | t
(2 rows)

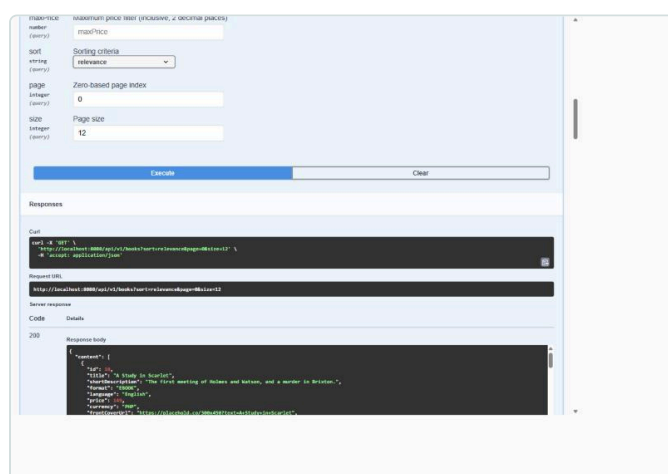
bookworm=> SELECT o.order_number, p.method, p.amount, p.status, p.card_last4, p.failure_reason, p.refunded AS NOT NULL AS refunded FROM payments p JOIN orders o ON o.id = p.order_id
order_number | method | amount | status | card_last4 | failure_reason | refunded
-----
BW-20261008-000005 | CREDIT_CARD | 670.88 | FAILED | 0002 | Card declined | f
BW-20261008-000005 | CREDIT_CARD | 670.88 | REFUNDED | 4242 | | t
BW-20261008-000006 | E_WALLET | 665.88 | REFUNDED | | | t
(3 rows)

bookworm=> SELECT o.order_number, g.type, g.points FROM gift_point_transactions g JOIN orders o ON o.id = g.order_id WHERE <this customer>;
order_number | type | points
-----
BW-20261008-000005 | EARNED | 6
BW-20261008-000006 | REDEEMED | -5
BW-20261008-000006 | EARNED | 6
BW-20261008-000006 | RESTORED | 5
BW-20261008-000006 | REVERSED | -6
BW-20261008-000005 | REVERSED | -6
(6 rows)

bookworm=> SELECT u.email, u.gift_points_balance, a.city, a.province, a.zip_code FROM users u JOIN addresses a ON a.user_id = u.id WHERE <this customer>;
email | gift_points_balance | city | province | zip_code
-----
e2w-2026100810215@bookworm.test | 0 | Makati | Metro Manila | 1210
(1 row)

```

Orders, payments, the gift points ledger and the saved Philippine address for the test customer.



A live `GET /books` call from Swagger UI.

```

getty print 2
{
  "id": 1,
  "title": "Will we laggers",
  "description": "Will's bookmark novel of love and injustice in colonial Philippines.",
  "author": "Mariano",
  "language": "English",
  "format": "Book",
  "url": "http://plasmaid.co/newsletter-laggers",
  "bookcover": "http://plasmaid.co/newsletter-laggers",
  "author": {
    "id": 1,
    "name": "Josef Alon",
    "description": "Will's author",
    "url": "http://plasmaid.co/newsletter-laggers"
  },
  "tags": [
    "tag": "Laggers",
    "tag": "Mariano Publishing House",
    "tag": "Laggers",
    "tag": "Historical",
    "tag": "Historical"
  ],
  "genres": [
    "Classic",
    "Fiction",
    "Philippine literature"
  ],
  "availability": 30,
  "stock": 100,
  "availability": 100,
  "availability": "2018-06-15",
  "format": "Book",
  "availability": "2018-06-15",
  "availability": {
    "id": 1,
    "name": "Mariano Publishing House",
    "url": "http://plasmaid.co/newsletter-laggers"
  }
}

```

Book detail: author bio, genres, stock, copies sold, delivery date and other formats.

6. Testing, and the bugs live runs caught

`mvn clean install` runs **146 JUnit 5 tests, 0 failures**, on H2 in PostgreSQL mode, so no database is needed to build. They cover controllers (`@WebMvcTest` with `MockMvc`), services against the real migrations, the named database constraints (26 tests), the seed data, security, and the contract itself (valid OpenAPI, no drift, error codes match).

mvn clean install: 146 tests, 0 failures

JUnit 5 on H2 in PostgreSQL mode, run with `mvn clean install` on 2026-10-08. Build result: **BUILD SUCCESS**

TEST CLASS	TESTS	FAILURES	ERRORS	SKIPPED
<code>ApiErrorCodeContractTest</code>	1	0	0	0
<code>contractDriftTest</code>	1	0	0	0
<code>ebookstoreApplicationTests</code>	1	0	0	0
<code>openApiContractTest</code>	4	0	0	0
<code>SecurityConfigIntegrationTest</code>	5	0	0	0
<code>SwaggeruiIntegrationTest</code>	3	0	0	0
<code>controller.AuthControllerTest</code>	5	0	0	0
<code>controller.BookControllerTest</code>	8	0	0	0
<code>controller.CartControllerTest</code>	6	0	0	0
<code>controller.OrderAndAccountControllerTest</code>	14	0	0	0
<code>exception.GlobalExceptionHandlerTest</code>	6	0	0	0
<code>repository.EntityMappingTest</code>	4	0	0	0
<code>repository.SchemeConstraintTest</code>	26	0	0	0
<code>repository.SeedCatalogueTest</code>	9	0	0	0
<code>service.AuthServiceTest</code>	6	0	0	0
<code>service.BookServiceIntegrationTest</code>	13	0	0	0
<code>service.CartServiceIntegrationTest</code>	9	0	0	0
<code>service.OrderAndAccountServiceIntegrationTest</code>	9	0	0	0
<code>service.PaymentServiceIntegrationTest</code>	16	0	0	0

Test classes and counts from the final build.

The most useful lesson: green tests were not enough. When all features were built, 141 tests passed.

Running the real app against PostgreSQL then found bugs that every test missed:

1. **A declined card was accepted.** The decline rule from spec 0002 had never been built.
2. **Cancel did not save.** A bulk stock update cleared the order change before it was written, so the order stayed CONFIRMED and a second cancel restored stock and points twice.
3. After Bob's first fix, **an order with a declined attempt broke the order history** (two payment rows where the code expected one), and **copies sold never changed**.

Each round went the same way: Claude Code found the bug in the live run and traced its cause in the code, I passed a precise fix request to Bob, Bob fixed it with a regression test that reads the database back, and Claude Code ran everything again. After two rounds: 146 tests and 40 of 40 live calls.

7. Lessons and limits

What worked

- **Specs before code.** Writing each decision down, with numbered acceptance criteria and the source of every value, let a second tool build from it without guessing, and let me check its work against something fixed.
- **A second model reviewing every spec** caught design bugs (race conditions, test database type mismatches) before they became code.
- **Splitting roles between two tools** worked once both shared the same repo rules: one designed and verified, the other built fast.
- **Running the real app** is what caught the bugs the generated tests did not.

What I would do differently

- Ask for database read back tests from the start, not only after a bug.
- Verify each feature on the running app as it lands, instead of marking several done after green builds and verifying at the end.
- Review more of the auto picked design decisions myself instead of leaving them for later.

Known limits

- Local run only; no Docker or cloud deployment.
- Payments are simulated (a card ending in 0002 is declined); no real gateway.
- No rate limiting on login, and no refresh tokens (the JWT lasts 60 minutes).
- Related books and recommendations are computed in memory, fine for 27 books but not for a large catalogue.
- Wishlist, reviews, coupons, bestsellers, admin and shipping are deferred.

How to run it

1. Create the PostgreSQL role and database as in `README.md`, and set `DB_USERNAME` and `DB_PASSWORD`.
2. `mvn clean install` (146 tests), then `mvn spring-boot:run`.
3. Open `http://localhost:8080/swagger-ui.html`, or import `docs/insomnia-collection.json` into Insomnia and run folders 1 to 6.

Sources: `docs/scope/scope.md`, `docs/specs/0001` to `0005`, `docs/ai-usage-log.md`, `docs/screenshots/` and the git history of the repository at the time of writing.